How gcc-Linux-implementation works for code-injection-detection
https://github.com/code-injection-detection/gcc-Linux-Implementation
14-March-2016

This is an unofficial guide. Many things are temporary and about to change. Also, this is a high-level approch and we do not delve into the details.

Ok let's start:

Directory structure:
1) src/: Inside there are the java source files than manipulate the assembly and the final executable
2) code/: In there are the other files that emulate the secure machine.
3) bin/: This directory does not exist on github. In there there are the java class files (that Eclipse produces automatically). Someone that does not use Eclipse should compile the .java files in the src/ directory and move them inside the bin/ directory. To save the hassle, a script does it automatically, if the java compiler (javac) is installed.

Now let's talk about the procedure that is followed.
First, we have a .c file that contains ALL of the code (except the functions of the standard C library that it #includes). That code is spread among different C files which are included by that main file. That code includes the main_program_function() function (that's practically the main, except some crypto initializations, so the actual main() has to be somewhere else), test functions, the secure heap implementation, the secure stack implementation and the verification procedure.
That program is compiled in its final “secure” form after some changes are done to it. So we actually work with the template of that program, to which we make changes and then compile.
Let's open **code/template_files/** directory and take a look at **main_program_template.c.** After the secure globals declaration (take a look at the python calls), we can see that it includes a file called “functions_needed_header.c”. After that there is the main_program_function() and then another file is included, the “functions_needed_footer.c”. These two files include every other function that may be needed. If we want to peek inside, we have to open **functions_needed_header_template.c** and **functions_needed_footer_template.c** .
That is not necessary right now. What needs to be understood are two things: a) Inside our template files there are calls to python scripts that change the code, so the final program that is compiled is the output of the python scripts (which pad the templates with proper C code), and b) Everything that is to be secured is in one compilation unit (since we use all those includes).

Now let's delve deeper into the securing procedure, explaining it step by step. The core of that procedure is a bash script, **code/automate.sh** . What it does in summary: Takes the user parameters, changes the templates using python, compiles the program into assembly, pads NOPS among the assembly commands (more on that later), assembles into executable, and changes the NOPS with keys and macs inside the executable. Please run it with parameters "./automate.sh 32 7 3 8 250000 8 20000 8 16" and wait for it to finish.

Let's open it and see it. In the start we can see the parameter fetching,some error checking and compilation of the java files if they are out of date. Then we see a line which starts with "python3 **set_correct_defines.py**". Let's go ahead and open that python file, inside **code/** directory. It consists of two parts. The first one sets the proper parameters in the file "headers_needed.h", and the second one is obsolete (is used for the former stack implementation, so we don't have to pay special attention to it). The file **template_files/headers_needed_template.h** is a header file included by almost every file in the project. It has many compile time parameters (like the size of the heap and the stack, the number of the keys etc). As we can see, the python script alters some of these parameters, as the user requests. It writes to the file "headers_needed.h" inside the **code/** directory. After that, every unit includes the non-template file "headers_needed.h" with the proper parameters.

Let's move on inside the bash script now. We see that the file **insert_new_stack_commands.py** is called (or its sister file, the **insert_new_stack_commands_groups_of_vars_as_arrays.py**). These are files that pad the C code that manipulates the secure stack. For example the file **template_files/tests_with_new_stack_template.c** makes a call " //HEY PYTHON CALLING FUNCTION : sieve_of_Eratosthenes_sec | PARAMETERS TO CALL WITH: 20000000 ". That call is parsed by python and replaced with the proper code that calls that secure function. The code that is replaced can be seen in the non-template file inside **code/** directory. If we open "tests_with_new_stack.c" we see that much code has been padded by the python script. Now, this file is in its "secure" form and it is ready to be included by the main program.

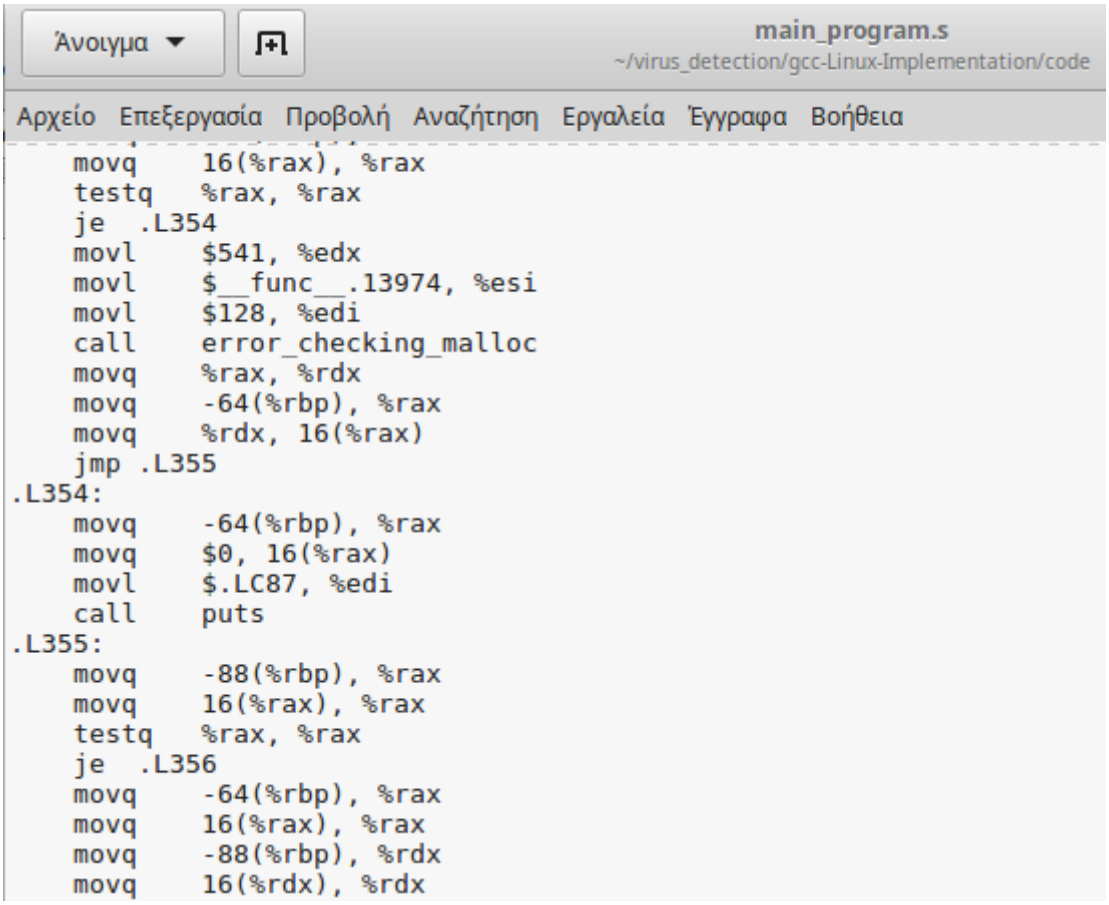
Moving on inside automate.sh, we see that some template files are copied inside **code/** directory. These files do not have any code that needs to be altered, and are simply copied from their template form to their usable form.

Next, some compilation takes place. Some files (like the implementation of the MAC, the secure getters and setters from the memory, the crypto initialization functions, stack allocation and deallocation etc), do not need to be secured. In fact, we assume that they are implemented in the hardware inside our secure CPU. So, their code should be a) Not secured, b) Optimized as much as possible to get close to their hardware implementation. We create their object files, in order for them to be able to be linked along with the secure code.

The next step is to initialize the global variables. At the start of the **template_files/main_program_template.c** we can see some global declaration. So, the next python script **insert_keys_and_macs_among_globals.py** does what its name says. We can see its output if we open the **code/main_program.c** and look at its start. Among the globals there are keys and macs.

After that we have some more cp's ,secure getters and setters compilation, and the main compilation of the program. To see what happens there, see the **Makefile**. It compiles the main program (with all of its #includes) into an assembly file, and then it compiles the assembly file into an executable, which serves as the baseline for our benchmarks. That executable contains code that does not use the secure getters/setters and its code is not secured, so it can give us benchmarks for the normal execution time.

However, after that stage we have an assembly file that some parts of it (which are to be benchmarked) use secure operations. It looks like this:




```
    movq    16(%rax), %rax
    testq   %rax, %rax
    je  .L354
    movl    $541, %edx
    movl    $__func__.13974, %esi
    movl    $128, %edi
    call    error_checking_malloc
    movq    %rax, %rdx
    movq    -64(%rbp), %rax
    movq    %rdx, 16(%rax)
    jmp .L355
.L354:
    movq    -64(%rbp), %rax
    movq    $0, 16(%rax)
    movl    $.LC87, %edi
    call    puts
.L355:
    movq    -88(%rbp), %rax
    movq    16(%rax), %rax
    testq   %rax, %rax
    je  .L356
    movq    -64(%rbp), %rax
    movq    16(%rax), %rax
    movq    -88(%rbp), %rdx
    movq    16(%rdx), %rdx
```

Now we approach the final stages of the automate.sh . The next step is to secure the assembly code. That is the job of the first java program. It (**Secure_Assembly.java** in **src/** directory) a) changes that assembly into an assembly that has NOP instructions inserted among groups of regular instructions. For example, if 7 NOPs are inserted between groups of 1 regular instruction, the new assembly file looks like: (Note: the picture is old, now the name is main_program_sec.s and code for verification is added)

set_correct_defines.py | Helloworldadd.s | Helloworldadd_sec.s

```
NOP
NOP
.UNIQUE8537:
    call    set_stack_int_array_element
.L679:
 jmp .UNIQUE8538
NOP
NOP
NOP
NOP
NOP
NOP
NOP
.UNIQUE8538:
    movq    -24(%rbp), %rax
 jmp .UNIQUE8539
NOP
NOP
NOP
NOP
NOP
NOP
NOP
.UNIQUE8539:
    movq    16(%rax), %rax
```

apart from that, b)this program adds code for on the fly verification of the code at the start of each block.

In our execution of the automate.sh script, we have 32 nops for keys , 16 nops for macs and some more nops for control, the assembly looks like (the nops are many): (Note: the first 4 lines are for on-the-fly code verification, they are not added by the compiler).

Άνοιγμα ▾

main_program_sec.s
~/virus_detection/gcc-Linux-Implementation/code

Αρχείο Επεξεργασία Προβολή Αναζήτηση Εργαλεία Έγγραφα Βοήθεια

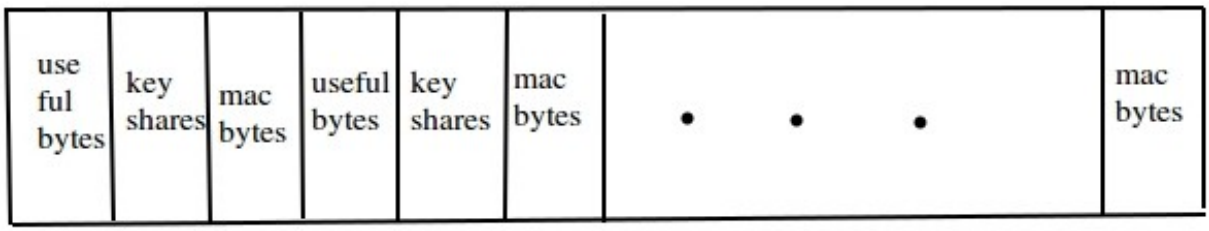
```
NOP
NOP
NOP
NOP
.UNIQUE2:
pushfq
movb $42,num_of_useful_bytes_to_mac_in_code(%rip)
call do_verify_code_on_the_fly
popfq
    movq    %rax, -8(%rbp)
    cmpq    $0, -8(%rbp)
    jne .L4
    movq    stderr(%rip), %rax
    movl    -36(%rbp), %ecx
    movq    -32(%rbp), %rdx
    movl    $.LC0, %esi
 jmp .UNIQUE3
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
```

Now we have an assembly file with padded NOPs after every X instructions (note: some instructions-calls and jmps- force creation of new blocks), and before these X instructions we have the verification code. The next part of the automate.sh script assembles that code and produces the executable.

The only thing that is left is to change the NOPs with the keys and macs that should be among the code lines. That is the job of **Secure_Machine_Code.java** in **src/** directory. After it is finished, we have the final, secure executable.

How secure memory is implemented

This section describes the way we implement the memory structure in the data segment, the heap and the stack (not the code segment). As the paper suggests, we interleave keys and macs among the useful bytes in memory. A picture is worth a thousand words:



The mac bytes contain the mac of the concatenation of <useful bytes>||<keyshares>.
As far as sizes are concerned, the keyshares are 32 bytes (2x 128 bit keys). The mac bytes are the output of AES-CBC-MAC. So, as the last block is 16 bytes (128 bits), the size of the mac is 16 bytes too. The useful bytes are variable. However, there are some things to consider.
a)When we verify the memory on the fly, we need to calculate as few CBC-MAC rounds as possible, so the less blocks we calculate the best. If we need 1 char (1 byte) from a useful byte area and the useful byte group is 20 bytes (or, more generally>16 bytes), then the mac will have to be calculated for about (20+32) div 16 =4 rounds. And all that for just 1 character. However, if the block is <=16 bytes, then we do the job with 3 rounds.
b)The secure CBC-MAC implementation requires adding the length of the data to be maced in the front, so we use 1 byte for that. Also, libcrypto's implementation of CBC-MAC uses 1 extra block for padding if it has to mac bytes whose length is multiple of 16. So, to achieve mac calculation with 3 blocks, we need the useful bytes to be maximum 14.
c) We use a specific way of storing variables into the secure memory. If we have to store an integer (4 bytes), we use an entire useful bytes block for it. Same for every other variable(chars, longs, etc), except for arrays. We allocate just enough useful byte groups to store the array, and its elements are one after another in the useful byte groups. So, if the useful bytes length is 6 bytes and we have an array of 2 integers, the first integer will be stored in the first useful byte block, and the second one will be split between the first and second useful byte groups:

<int1_byte1><<int1_byte2><int1_byte3><int1_byte4><int2_byte1><int2_byte2><k

eys><mac> <int2_byte3><int2_byte4><byte><byte><keys><mac>.

So, in case we need to fetch the second integer, we have to verify both the first and the second mac, which adds a lot to the delay.
That leads to the conlusion that 14 useful bytes may cause longs and pointers (8 bytes in 64 bit architecture) to be split between useful byte blocks. So, the optimal size of the useful bytes group is 8 bytes long, which causes no splits.

The way to access the secure memory is through secure getters and setters. Functions like “void set_stack_int(void * start_of_secure_data,int source)” are used to store data to the secure stack, heap or the secure globals. We have implemented secure getters/setters for a subset of the entire C functionality (our subset is still strong enough to be able to describe most common problems). More specifically, we have getters and setters for chars,ints,long ints, floats,doubles, pointers and their arrays. Furthermore, we have a way to store an arbitrarily-sized data structure into the secure memory, which is fetched from the unsecured memory (and of course the inverse, fetched from the secure memory and stored into the unsecured).

<u>How the globals are implemented</u>

The secure global variables all reside in a struct. Please open **template_files/main_program_template.c** . At its start the global declaration is obvious. When we need to declare a global, we first add a directive to the python scripts and then denote the size of the global. Python parses our commands and then initializes the variables, adding keys and macs after them in the next struct (which is empty in the template). Please open “main_program.c” in “code” directory. If we take a look inside it, we can see the globals,the keys and the macs, and after them their initial values.
The secure getters and setters are straightforward (see the last lines of **template_files/secure_getters_setters_template.h**). We have the standard type of getters, and generic setters for normal blocks of code and for for loops. You can see one example of secure global access in the “void secure_global_sum_calculator(int times, int maxval)” function inside **template_files/general_tests_template.c.**

/**********READ UNTIL HERE**********/
/**********THE REST IS NOT FINISHED **********/

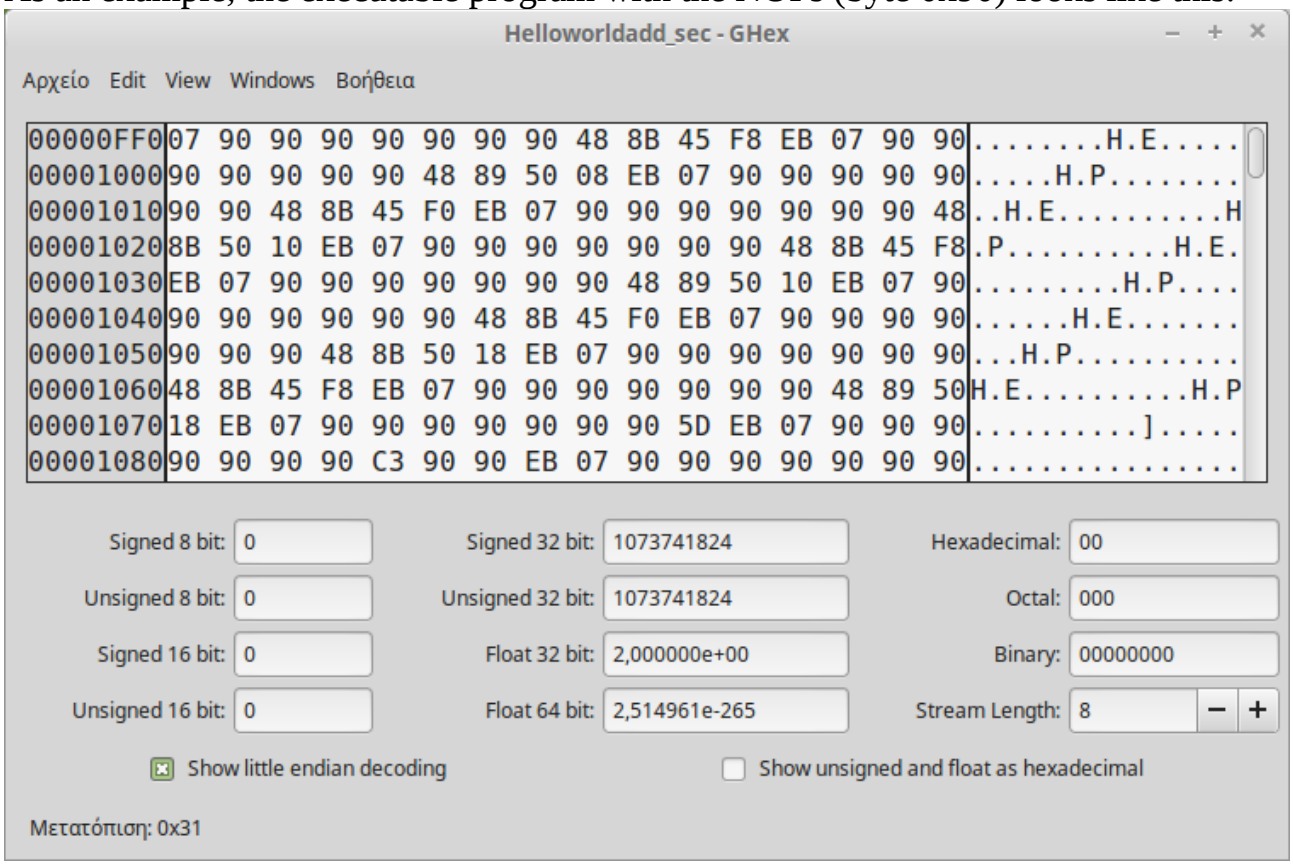
Now a java program (Secure_Assembly.java) will change that assembly into an assembly that has NOP instructions inserted among groups of regular instructions. For example, if 7 NOPs are inserted between groups of 1 regular instruction, the new assembly file looks like:

In this example, the 7 NOPs correspond to 5 places for the keys to be put (the final 5 NOPs) , and 2 places for canary values that actually depict that “hey, keys are following”, in order to help the verification procedure.

That assembly file is then assembled into an executable, without any optimizations (so as not to remove the NOPs or the command order).

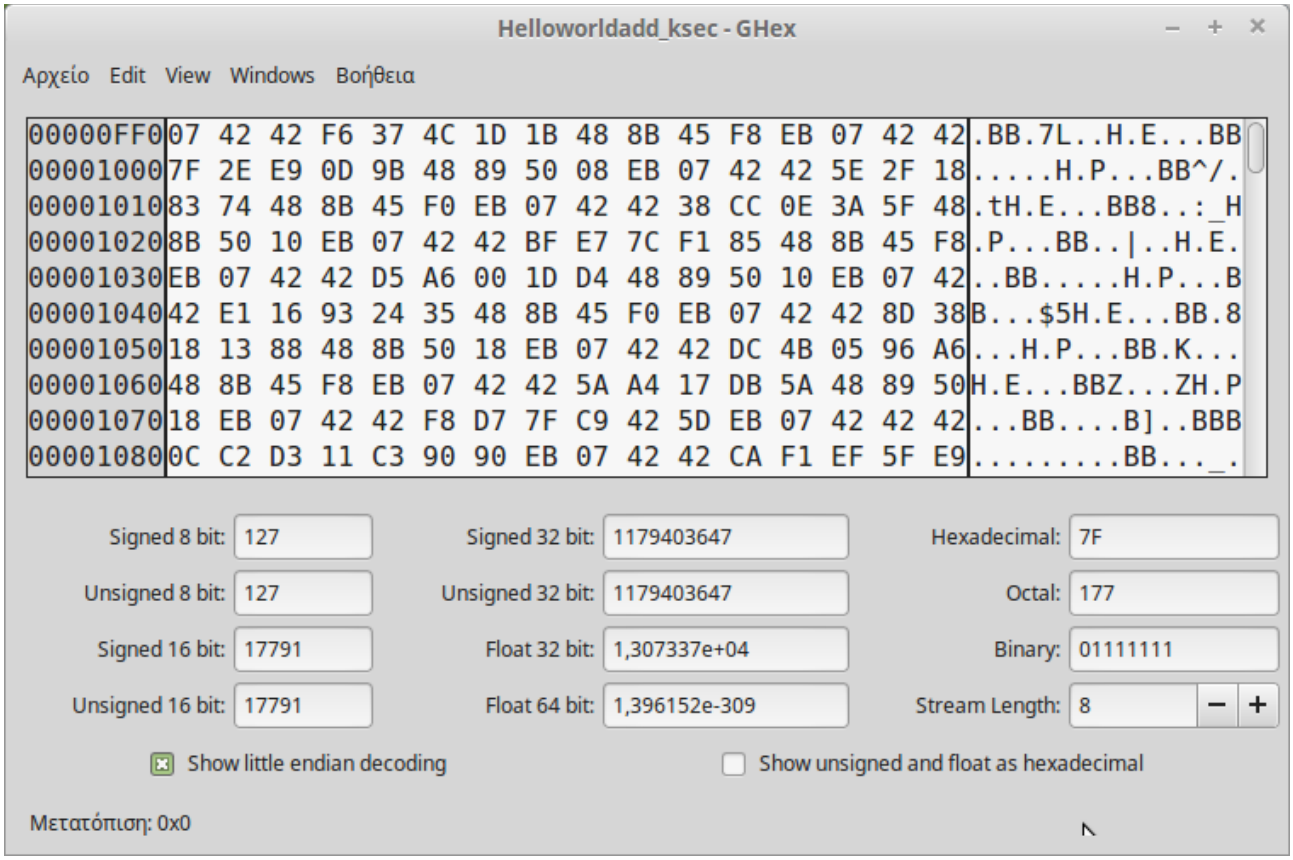
After that, another java program (Secure_Machine_Code.java) takes the produced binary, searches it byte-by-byte to find the pattern “jump +7 bytes and 7 NOPs are following” and replaces the NOP bytes with canaries and keyshares. The same java program also produces keyshares that will be inserted into the secure heap and secure stack. These ones are saved into the corresponding files. Finally, it outputs the XOR of all the keyshares, that are the keys that must be matched by the verification procedure.

As an example, the executable program with the NOPs (byte 0x90) looks like this:



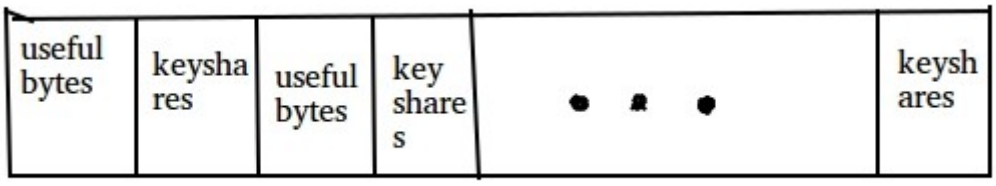
The JMP 07 instruction (EB 07) is clear, with the NOPs that follow it.

Same program with the NOPs replaced. The 2 canary values are 2 bytes of 0x42:



This is the final executable. When it runs, it initialises secure heap and stack (using the files with the keyshares), installs a signal handler for SIGUSR1 which, when invoked, starts the verification procedure, and then continues execution to perform the tests. Update: Along with the signal handler, it starts a listening socket in order to accept inbound connections from the outside that seek to verify the code.

The secure heap and secure stack are structured in the following way:



Important notes: a)The MACs have not been added yet.
b)The “useful bytes” chunks are same sized. The same applies to the keyshares chunks.
c)Every access to the secure stack or heap is done through the proper secure_* functions.

The verification procedure starts from the start of the code, and up until the end of the code searches for the sequence “EB 07 42 42” (in this example, not in every case), and calculates the keyshares from the bytes after that sequence. After that it fetches the keyshares from the secure heap and the secure stack. It XORs all the bytes and presents the final keys, which must be the same with the ones printed by the second java program.

Now let's take a glance at some of the most important files.

automate.sh: It's the bash script that does all of the preprocessing , given some parameters by the user.
a)It transforms the headers with the parameters that the user requested (for example the size of the secure heap)
b)It transforms directives inside .c files that the user has written, into valid C code. These directives are about the secure stack. a) and b) are performed with a python script.
c)Compiles the program into assembly.
d)Adds NOPs to assembly (with java)
e)Assembles the program to create an executable
f)Replaces NOPs with keys (with java)

set_correct_defines.py: This python program changes the defined values inside the headers_needed_template.h file (before the compilation), and parses the user's directives inside the general_tests_template.c file. With these, it produces valid C code.

(Inside templates directory:)
***template.{c,h}**: These are the template files that a programmer should edit. If a program X.{c,h} has a corresponding X_template.{c,h}, this means that the programmer should edit the template. The X.{c,h} program is used for the actual compilation, but that is after the code of the corresponding template is copied over it, and it is processed by the above python program. So, any changes to the X.{c,h} program will be OVERWRITTEN next time the automate.sh script is invoked. Update: Now the templates are in a separate directory.

Helloworldadd.c : This is the main program. It's name comes from the “HelloWorld” and an “add” test function. Its name is awful and will be changed. All it does is test the secure implementation. It includes the “.c” files of the secure heap and stack, the test files and the verification procedure. All these are in the “functions_needed.c” file. It includes C files and not .h files because it is easier for the java program that inserts NOPs afterwards to deal with one and not multiple programs. After the includes and the initialization of the secure stuff, it performs the tests.

stack_manager_template.c
memory_manager_template.c
verification_procedure_template.c: They implement the secure stack and heap, and the verification procedure. More detail in a separate document.

memory_manager_test_suite.c
stack_manager_test_suite.c
general_tests_template.c: They have some tests. They are nothing important. Someone should only look there (especially in the general tests) to see how to invoke the secure functions. More detail in a separate document.

Makefile: The Makefile for the C programs. The interesting thing there is the FLAGS_to_assembly, in which optimizations can be put.

So, to end the high-level approach, let's demonstrate an example run:

Inside the main() function we have:

```
...
init_heap_and_stack_mem();
install_signal_handler();
...
int hanoi_up_to=25;
TEST_NORMAL_AND_SECURE_TIME( towers of Hanoi, towerOfHanoi(hanoi_up_to,'A','C','B');
,towerOfHanoi_secure_using_template( hanoi_up_to,'A','C','B'); )
printf("\n");
printf("Going to find keyshares.\n");
find_keyshares(); //for verification
free_heap_and_stack_memory();
```

The TEST_NORMAL_AND_SECURE_TIME is a macro that does what it says.

We can invoke the automate.sh script with the following parameters:

```
./automate.sh <a> <b> <c> <d> <e> <f> <g>
Where: a=number_of_interleaved keys in code, b=number_of grouped instructions in code
c=number of canary values before keyshares in code, to denote keyshare presence
d=number of grouped useful bytes in heap memory, e=total bytes to (try to) pre-allocate in heap memory
f=number of grouped useful bytes in stack memory, g=total bytes to (try to) pre-allocate in stack memory
Example: ./automate.sh 5 1 2 4 2048 4 1024
```

For the hanoi towers we will give some more stack memory. Lets say 8192 bytes.

```
./automate.sh 5 1 2 4 2048 4 8192
NUM_OF_INTERLEAVED_KEYS: 5
NUM_OF_GROUPED_INSTRUCTIONS: 1
NUM_OF_GROUPED_USEFUL_BYTES: 4
```

```
NUM_OF_TOTAL_BYTES_ALLOC: 2048
NUM_OF_CANARIES: 2
NUM_OF_GROUPED_USEFUL_STACK_BYTES: 4
NUM_OF_TOTAL_STACK_BYTES_ALLOC: 8192

Changing defines according to input...
Changed defines.
Copying templates to target files
Copied templates
Compiling....
gcc -O0 Helloworldadd.c -S -lm
gcc -O0 Helloworldadd.s -c -o Helloworldadd.o -lm
gcc -O0 Helloworldadd.o -o Helloworldadd -lm
Compiled.
Inserting NOPs into assembly...
NOPs inserted.
Assembling code with NOPs...
gcc -O0 Helloworldadd_sec.s -c -o Helloworldadd_sec.o -lm
gcc -O0 Helloworldadd_sec.o -o Helloworldadd_sec -lm
Assembled.
Replacing NOPs with keys...

Keyshare 0 : 0xF2
Keyshare 1 : 0xF5
Keyshare 2 : 0xA5
Keyshare 3 : 0x77
Keyshare 4 : 0x62
Number of nop groups replaced: 9999
NOPs replaced with keys.
```

And running the executable will yield:

```
./Helloworldadd_ksec
Initializing heap memory
bytes_to_allocate_on_start:2048
Init_mem, alloc+key insertion
Whoops. We need to allocate a bit more space.
If successful, total bytes allocated:2056
Initializing stack memory
Stack bytes_to_allocate_on_start:8192
Init_stack_mem, alloc+key insertion
Stack: Whoops. We need to allocate a bit more space.
If successful, total bytes allocated:8194
Installing signal handler
Signal handler installed
My pid=13566
K=6
n=10
main is at 0x420a31

Calculating time for towers of Hanoi. Normal towers of Hanoi:

Normal towers of Hanoi:0.504955 cpu seconds
Secure towers of Hanoi:

Normal towers of Hanoi time:0.504955 cpu seconds
Secure towers of Hanoi time:29.8955 cpu seconds
```

Ratio: 59.2042 times slowdown

Going to find keyshares.

key no0=0xf2
key no1=0xf5
key no2=0xa5
key no3=0x77
key no4=0x62